

為網頁應用程式打造即時導覽體驗和流暢頁面轉場

來源：<https://codelabs.developers.google.com/create-an-instant-and-seamless-web-app?hl=zh-tw>

步驟數：7

瞭解如何使用 Google Chrome 的最新 API，在網頁應用程式中加入即時導覽和流暢的頁面轉場功能。

目錄

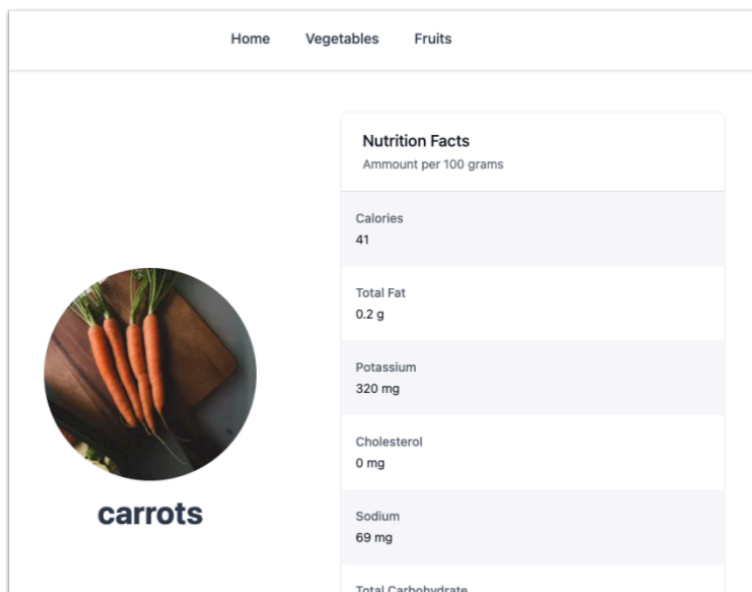
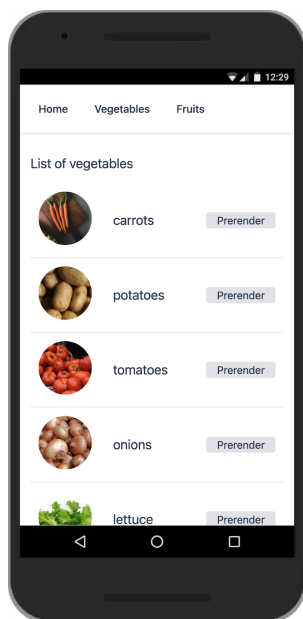
1. [1. 事前準備](#)
2. [2. 開始使用](#)
3. [3. 整合預先算繪功能](#)
4. [4. 移除 bfcache 阻礙因素](#)
5. [5. 啟用跨網站預先擷取](#)
6. [6. 整合 Shared Element Transitions API](#)
7. [7. 恭喜](#)

步驟 1 · 約 5 分鐘

1. 事前準備

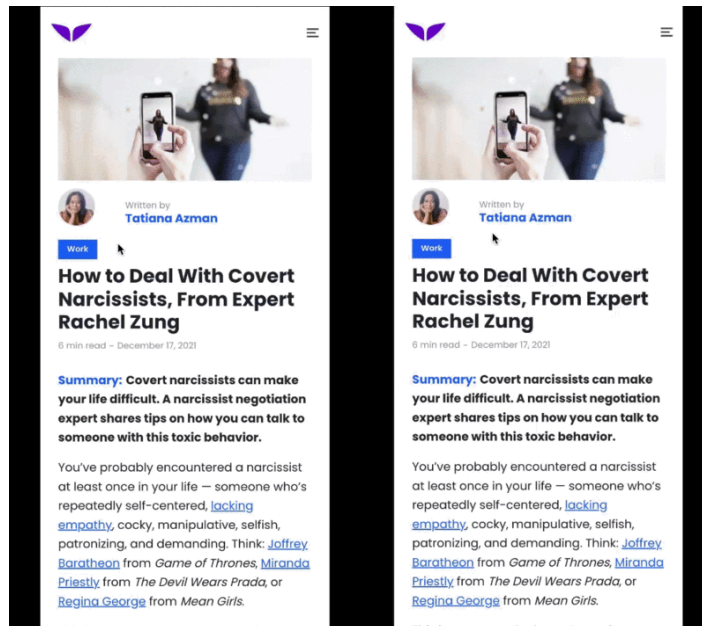
本程式碼研究室會說明如何使用 Google Chrome 原生支援的最新 API，為範例網頁應用程式新增即時導覽體驗和流暢頁面轉場效果。

這個範例網頁應用程式會檢查熱門水果和蔬菜的營養價值。水果清單和水果詳細資料頁面是做為單頁應用程式 (SPA) 建構，蔬菜清單和蔬菜詳細資料頁面則是做為傳統多頁應用程式 (MPA) 建構。



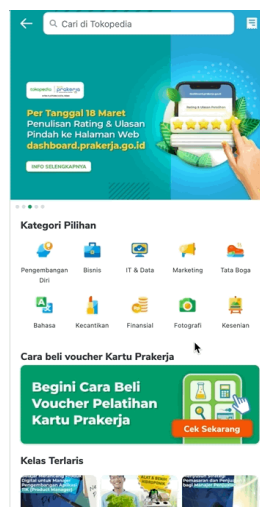
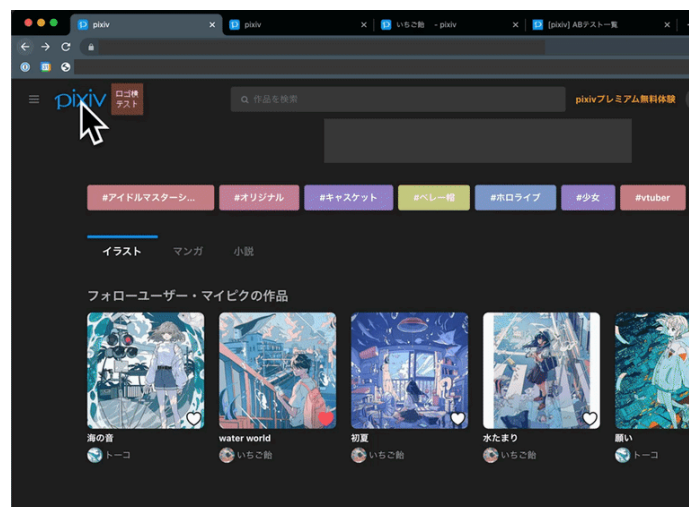
具體來說，您會實作[預先算繪](#)、[往返快取 \(bfcache\)](#) 和 [私密預先擷取 Proxy](#)，實現即時導覽，並實作[根/共用元素轉場效果](#)，實現無縫網頁轉場效果。您為 MPA 網頁導入預先算繪和 bfcache，並為 SPA 網頁導入共用元素轉換。

網站速度一直是使用者體驗的重要環節，因此 Google 推出了 [Core Web Vitals](#)，這是一組衡量網頁載入效能、互動性和視覺穩定性的指標，可評估實際使用者體驗。最新的 API 可協助您改善網站的現場 Core Web Vitals 分數，特別是載入效能。



Mindvalley 的示範

使用者也習慣在行動原生應用程式中使用轉場效果，讓導覽和狀態變更極為直覺。很遺憾，在網路上複製這類使用者體驗並不容易。雖然您或許能使用目前的網頁平台 API 達成類似效果，但開發過程可能過於困難或複雜，尤其是與 Android 或 iOS 應用程式中的對應功能相比時。無縫 API 的設計宗旨，是填補應用程式和網站之間的使用者與開發人員體驗落差。



pixiv和 Tokopedia 的示範

注意：這些 API 仍處於早期階段，因此可能會有錯誤、限制和缺少功能。此外，API 介面日後可能會變更。如要提供意見或回報錯誤，請前往 crbug.com。

必要條件

瞭解：

- HTML
- CSS
- JavaScript
- [Google Chrome 開發人員工具](#)

報告內容：

實作方式：

- 預先算繪
- bfcache
- Private Prefetch Proxy
- 根/共用元素轉換

建構項目

以 Next.js 建構的範例網頁應用程式，並採用最新的即時無縫瀏覽器功能：

- 透過預先算繪功能，近乎即時地導覽
- 往返快取，可透過瀏覽器的上一頁和下一頁按鈕即時載入網頁
- 透過 Private Prefetch Proxy 或 Signed Exchange (SXG) 進行跨來源導覽，營造良好的第一印象
- 使用根/共用元素轉場，在頁面之間流暢轉場

軟硬體需求

- Chrome 101 以上版本
-

步驟 2 · 約 2 分鐘

2. 開始使用

啟用 Chrome 旗標

1. 前往 `about://flags`，然後啟用 `Prerender2` 和 `documentTransition API` 執行階段標記。
2. 重新啟動瀏覽器。

取得程式碼

1. 在您慣用的開發環境中，開啟[這個 GitHub 存放區](#)中的程式碼：

```
git clone -b codelab git@github.com:googlechromelabs/instant-seamless-demo.git
```

2. 安裝執行伺服器所需的依附元件：

```
npm install
```

3. 在通訊埠 3000 啟動伺服器：

```
npm run dev
```

4. 在瀏覽器中前往 <http://localhost:3000>。

現在您可以編輯及改善應用程式。每當您進行變更時，應用程式會重新載入，並直接顯示變更。

3. 整合預先算繪功能

為了進行這項示範，範例應用程式中蔬菜詳細資料頁面的載入時間非常緩慢，這是因為伺服器端出現任意延遲。預先算繪可消除這段等待時間。

如要在蔬菜清單頁面中新增預先算繪按鈕，並在使用者點擊後觸發預先算繪：

1. 建立按鈕元件，動態插入 `speculation-rules` 指令碼標記：

`components/prerender-button.js`

```
import { useContext } from 'react'
import ResourceContext from '../resource-context'

// You use resource context to manage global states.
// In the PrerenderButton component, you update the prerenderURL parameter when the button is
// clicked.
export default function PrerenderButton() {
  const { dispatch } = useContext(ResourceContext)
  const handleClick = (e) => {
    e.preventDefault()
    e.stopPropagation()
    const parent = e.target.closest('a')
    if (!parent) {
      return
    }
    const href = parent.getAttribute('href')
    dispatch({ type: 'update', prerenderURL: href })
  }

  return (
    <button className='ml-auto bg-gray-200 hover:bg-gray-300 px-4 rounded' onClick=
{handleClick}>
      Prerender
    </button>
  )
}
```

2. 在 `list-item.js` 檔案中匯入 `PrerenderButton` 元件。

`components/list-item.js`

```
// Codelab: Add a PrerenderButton component.
import PrerenderButton from './prerender-button'

...

function ListItemForMPA({ item, href }) {
  return (
    <a href={href} className='block flex items-center'>
      <Icon src={item.image} />
      <div className='text-xl'>{item.name}</div>
      { /* Codelab: Add PrerenderButton component. */ }
      <PrerenderButton />
    </a>
  )
}
```

3. 建立元件，新增 Speculation Rules API。

當應用程式更新 `prerenderURL` 狀態時，`SpeculationRules` 元件會動態將指令碼標記插入網頁。

components/speculationrules.js

```
import Script from 'next/script'
import { useContext, useMemo } from 'react'
import ResourceContext from './resource-context'

export default function SpeculationRules() {
  const { state } = useContext(ResourceContext)
  const { prerenderURL } = state

  return useMemo(() => {
    return (
      <>
        {prerenderURL && (
          <Script id='speculationrules' type='speculationrules'>
            {`
            {
              "prerender":[
                {
                  "source": "list",
                  "urls": ["${prerenderURL}"]
                }
              ]
            }
            `}
          </Script>
        )}
      </>
    )
  }, [prerenderURL])
}
```


4. 將元件與應用程式整合。

pages/_app.js

```
// Codelab: Add the SpeculationRules component.
import SpeculationRules from '../components/speculationrules'

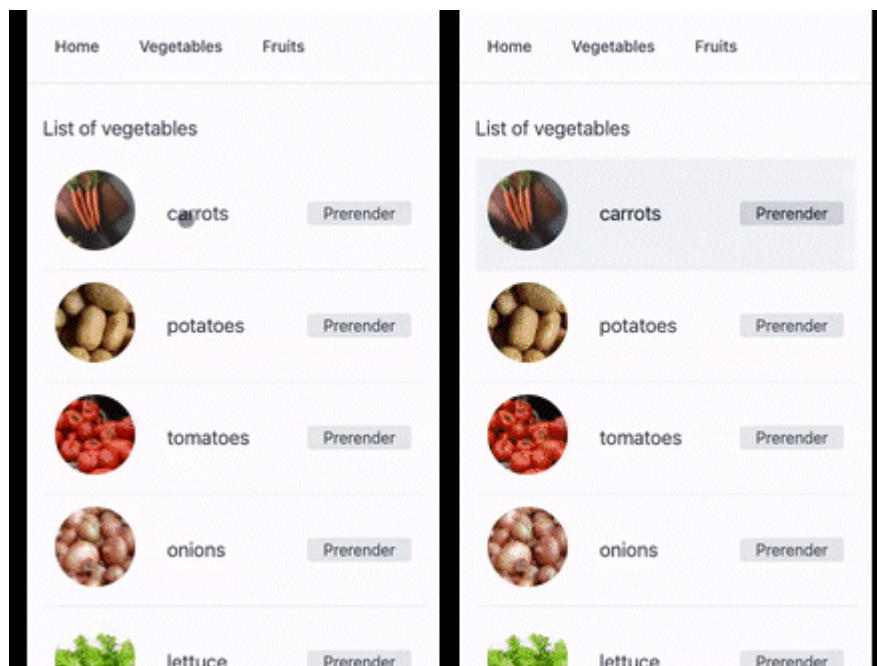
function MyApp({ Component, pageProps }) {
  useAnalyticsForSPA()

  return (
    <ResourceContextProvider>
      <Layout>
        <Component {...pageProps} />
      </Layout>
      { /* Codelab: Add SpeculationRules component */ }
      <SpeculationRules />
      <Script id='analytics-for-mpa' strategy='beforeInteractive' src='/analytics.js' />
    </ResourceContextProvider>
  )
}

export default MyApp
```

5. 按一下「預先算繪」。

現在您可以看到載入速度大幅提升。在實際用途中，系統會根據某些啟發式方法，為使用者可能接下來造訪的網頁觸發預先算繪。



Analytics

根據預設，範例網頁應用程式中的 `analytics.js` 檔案會在發生 `DOMContentLoaded` 事件時傳送網頁瀏覽事件。很抱歉，這並非明智之舉，因為這個事件會在預先算繪階段觸發。

如要導入 `document.prerendering` 和 `prerenderingchange` 事件來修正這個問題，請按照下列步驟操作：

- 重寫 `analytics.js` 檔案：

public/analytics.js

```
const sendEvent = (type = 'pageview') => {
  // Codelab: Make analytics prerendering compatible.
  // The pageshow event could happen in the prerendered page before activation.
  // The prerendered page should be handled by the prerenderingchange event.
  if (document.prerendering) {
    return
  }
  console.log(`Send ${type} event for MPA navigation.`)
  fetch(`/api/analytics?from=${encodeURIComponent(location.pathname)}&type=${type}`)
}
...

// Codelab: Make analytics prerendering compatible.
// The prerenderingchange event is triggered when the page is activated.
document.addEventListener('prerenderingchange', () => {
  console.log('The prerendered page was activated.')
  sendEvent()
})
...
```

太棒了！您已成功修改 Analytics，使其與預先算繪功能相容。現在您可以在瀏覽器控制台中，查看時間正確的網頁瀏覽記錄。

步驟 4 · 約 6 分鐘

4. 移除 bfcache 阻礙因素

移除 `unload` 事件處理常式

不必要的 `unload` 事件是很常見的錯誤，現在已不建議使用。這不僅會導致 bfcache 無法運作，而且也不可靠。舉例來說，在[行動裝置](#)和 [Safari](#) 上，這個事件不一定會觸發。

您可以使用 `pagehide` 事件，取代 `unload` 事件。`pagehide` 事件會在 `unload` 事件觸發時，以及網頁放入 bfcache 時觸發。

如要移除 `unload` 事件處理常式，請按照下列步驟操作：

- 在 `analytics.js` 檔案中，將 `unload` 事件處理常式的程式碼替換為 `pagehide` 事件處理常式的程式碼：

`public/analytics.js`

```
// Codelab: Remove the unload event handler for bfcache.
// The unload event handler prevents the content from being stored in bfcache. Use the
pagehide event instead.
window.addEventListener('pagehide', () => {
  sendEvent('leave')
})
```

更新快取控制項標頭

使用 `Cache-control: no-store` HTTP 標頭提供的網頁無法享有瀏覽器的 bfcache 功能，因此建議您謹慎使用這個標頭。特別是如果網頁不含個人化或重要資訊 (例如登入狀態)，您可能不需要使用 `Cache-control: no-store` HTTP 標頭提供該網頁。

注意：Chrome 團隊建議在某些情況下 (例如沒有 Cookie 變更時)，將 `Cache-Control: no-store` 內容儲存在 bfcache 中，以充分發揮 bfcache 的效能優勢。如要瞭解詳情及提供提案意見回饋，請參閱「[BFCache & Cache-control:no-store](#)」。

如要更新範例應用程式的快取控制標頭，請按照下列步驟操作：

- 修改 `getServerSideProps` 程式碼：

注意：使用 `next dev` 在本機執行應用程式時，系統會覆寫標頭，避免在本機快取 ([Next.js 文件](#))。覆寫的標頭為 `Cache-Control: no-cache, no-store, max-age=0, must-revalidate`。

如要測試網頁是否符合 bfcache 資格，您必須建立正式版，並檢查是否含有適當的快取標頭。

`pages/vegetables/index.js`

```
export const getServerSideProps = middleware(async (ctx) => {
  const { req, res } = ctx
  // Codelab: Modify the cache-control header.
  res.setHeader('Cache-Control', 'public, s-maxage=10, stale-while-revalidate=59')
  ...
})
```

pages/vegetables/[name].js

```
export const getServerSideProps = middleware(async (ctx) => {
  const { req, res, query } = ctx
  // Codelab: Modify the cache-control header.
  res.setHeader('Cache-Control', 'public, s-maxage=10, stale-while-revalidate=59')
  ...
})
```

判斷網頁是否從 bfcache 還原

網頁初次載入時，系統會在 `load` 事件後立即觸發 `pageshow` 事件，且每次從往返快取還原網頁時也會觸發。 `pageshow` 事件具有 `persisted` 屬性，如果網頁是從 bfcache 還原，這個屬性會是 `true`，否則為 `false`。您可以使用 `persisted` 屬性，區分一般網頁載入和 bfcache 還原。主要的 Analytics 服務應會注意到 bfcache，但您可以檢查網頁是否從 bfcache 還原，並手動傳送事件。

如要判斷網頁是否從 bfcache 還原，請按照下列步驟操作：

- 將這段程式碼新增至 `analytics.js` 檔案。

public/analytics.js

```
// Codelab: Use the pageshow event handler for bfcache.
window.addEventListener('pageshow', (e) => {
  // If the persisted flag exists, the page was restored from bfcache.
  if (e.persisted) {
    console.log('The page was restored from bfcache.')
    sendEvent()
  }
})
```

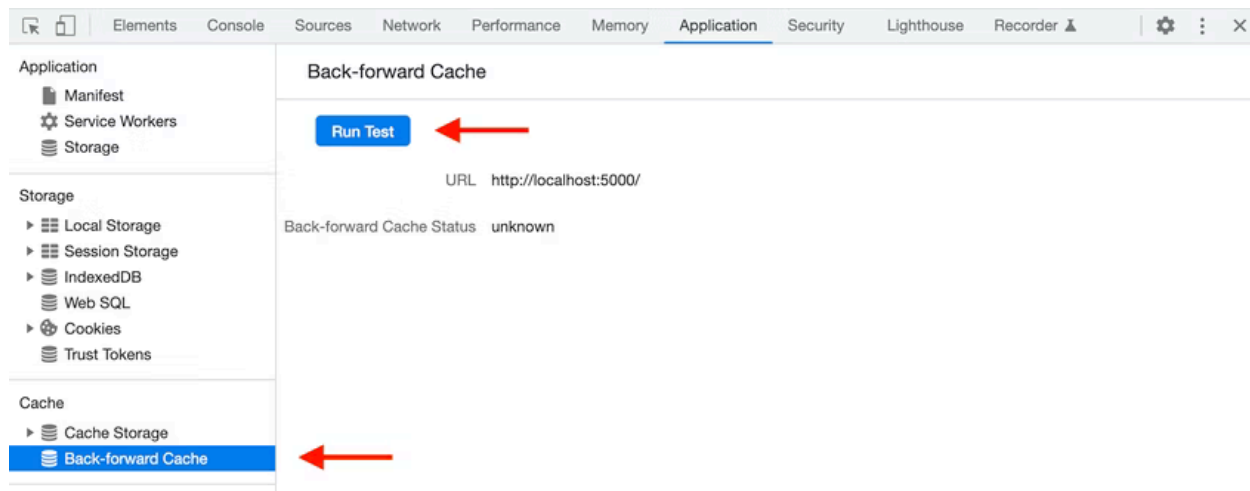
偵錯網頁

Chrome 開發人員工具可協助您測試網頁，確保網頁已針對 bfcache 最佳化，並找出可能導致網頁不符合資格的問題。

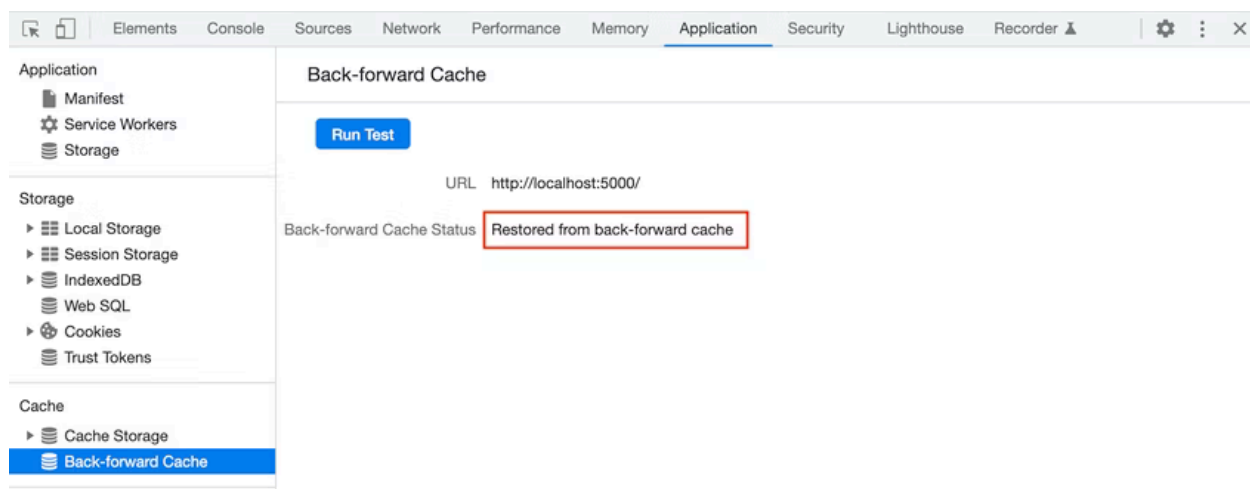
如要測試特定網頁：

- 在 Chrome 中前往該頁面。
- 在 Chrome 開發人員工具中，依序點選「應用程式」>「往返快取」>「執行測試」。

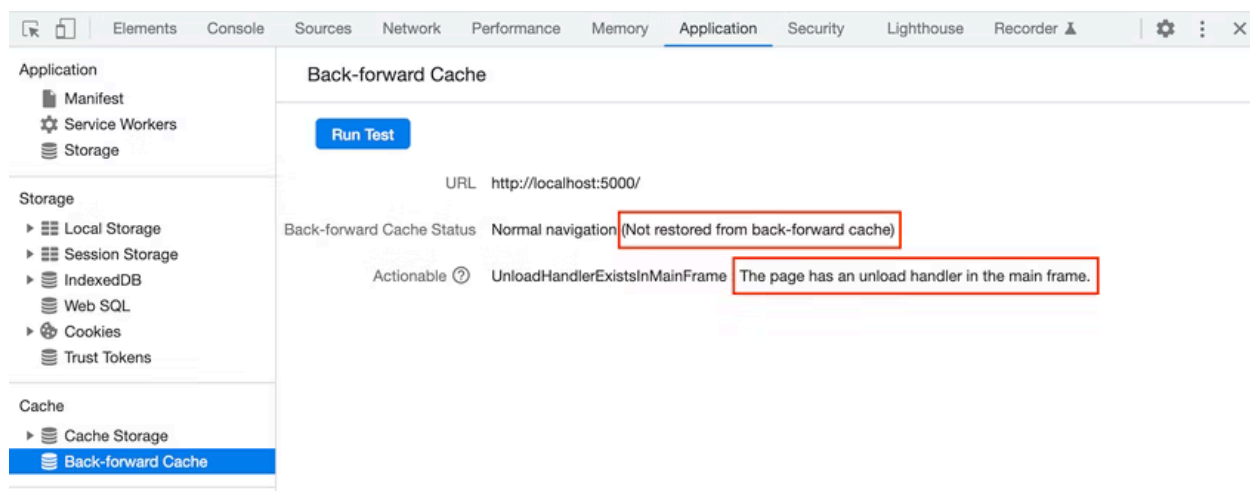
Chrome 開發人員工具會嘗試離開網頁，然後返回，判斷網頁是否可從 bfcache 還原。



如果成功，面板會顯示頁面已從往返快取還原：



如果還原失敗，面板會顯示網頁未還原，並說明原因。如果開發人員可以解決問題，面板也會顯示相關資訊。



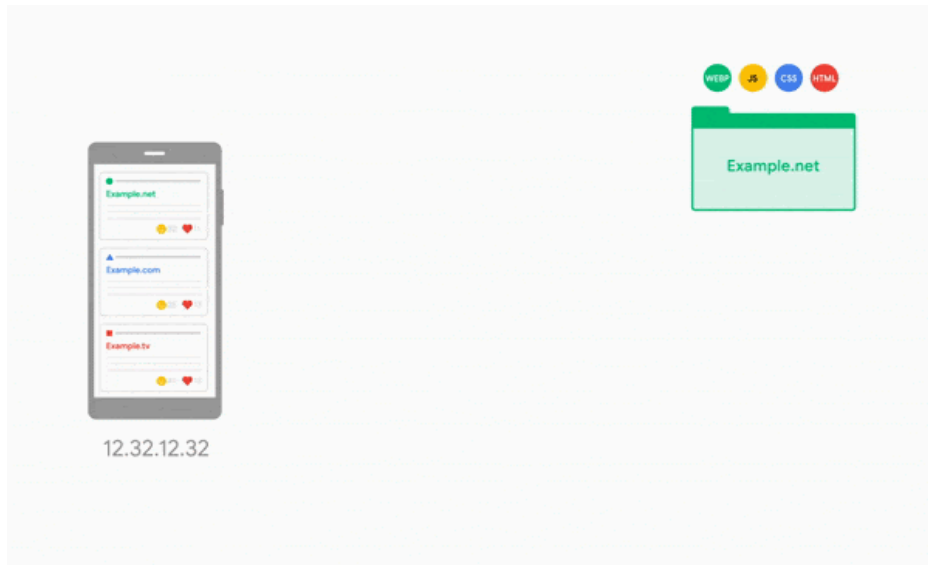
注意：在電腦上進行偵錯時，由於安裝的擴充功能會新增卸載處理常式和開啟連線，因此網頁通常會不符合 bfcache 的資格。使用無痕視窗可避免瀏覽器擴充功能造成非預期的干擾。

步驟 5 · 約 6 分鐘

5. 啟用跨網站預先擷取

預先擷取會提早開始擷取，讓位元組在使用者瀏覽時已位於瀏覽器中，進而加快瀏覽速度。這項功能可輕鬆改善 Core Web Vitals，並在導覽前抵銷部分網路活動。這會直接加快「[最大內容繪製](#)」(LCP)，並在導覽時為「[首次輸入延遲](#)」(FID) 和「[累計版面配置位移](#)」(CLS) 預留更多空間。

[私密預先擷取 Proxy](#) 可啟用跨網站預先擷取功能，但不會向目的地伺服器揭露使用者的私人資訊。



透過 Private Prefetch Proxy 啟用跨網站預先擷取

注意：發布本程式碼研究室時，Private Prefetch Proxy 屬於[搶先體驗計畫](#)，使用者必須選擇加入才能使用。如需最新資訊，請參閱 goo.gle/p3-info。

網站擁者可透過已知的 [traffic-advice](#) 資源控管預先擷取作業，這與網路檢索器的 `/robots.txt` 類似，可讓 HTTP 伺服器聲明實作代理程式應套用相應的建議。目前網站擁者可以建議代理程式禁止或限制網路連線。日後可能會新增其他建議。

如要代管交通資訊資源：

1. 新增類似 JSON 的檔案：

`public/.well-known/traffic-advice`

```
[
  {
    "user_agent": "prefetch-proxy",
    "google_prefetch_proxy_eap": {
      "fraction": 1
    }
  }
]
```

`google_prefetch_proxy_eap` 欄位是搶先體驗計畫的專用欄位，`fraction` 欄位則是用來控管 Private Prefetch Proxy 傳送的預先擷取要求比例。

流量建議應以 `application/trafficadvice+json` MIME 類型傳回。

2. 在 `next.config.js` 檔案中，設定回應標頭：

`next.config.js`

```
const nextConfig = {
  // Codelab: Modify content-type for traffic advice file.
  async headers() {
    return [
      {
        source: '/.well-known/traffic-advice',
        headers: [
          {
            key: 'Content-Type',
            value: 'application/trafficadvice+json',
          },
        ],
      },
    ],
  },
}

module.exports = nextConfig
```

注意：如要啟用跨網站預先擷取功能，也可以使用[Signed Exchange \(SXG\)](#)。我們提供一些實用程式庫和工具，方便您提供 SXG 檔案。詳情請參閱「[開始在 Google 搜尋中使用 Signed Exchange](#)」和「[Signed Exchange \(SXG\)](#)」。

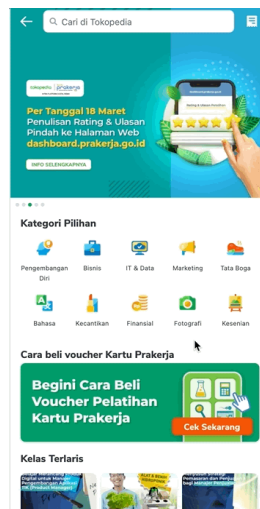
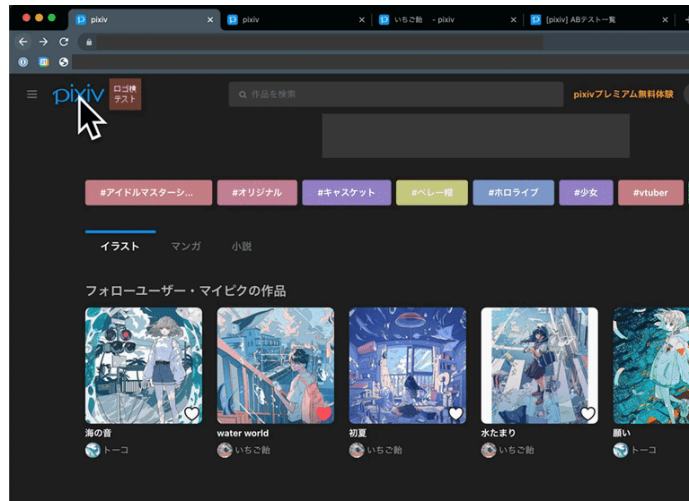
步驟 6 · 約 12 分鐘

6. 整合 Shared Element Transitions API

使用者在網頁上從一個頁面導覽至另一個頁面時，第一個頁面會消失，新頁面會顯示，因此使用者看到的內容會突然且意外地變更。這種不連貫的循序式使用者體驗會讓使用者感到困惑，且由於使用者必須拼湊出自己抵達目前位置的方式，因此會造成較高的認知負荷。此外，使用者等待載入所需目的地時，也會覺得網頁載入速度變快。

流暢的載入動畫可降低認知負擔，因為使用者在網頁間瀏覽時會保持在情境中，同時也會看到引人入勝的內容，因此可減少載入的感知延遲。因此，大多數平台都會提供簡單易用的基本元素，讓開發人員建構順暢的轉場效果，例如 Android、iOS、macOS 和 Windows。

[共用元素轉場效果 API](#) 可讓開發人員在網路上使用相同功能，無論轉場效果是跨文件 (MPA) 或文件內 (SPA) 皆適用。



[pixiv](#)和[Tokopedia](#) 的展示

注意：Shared Element Transitions API 仍處於非常早期的階段，且在發布本程式碼研究室時，僅支援 SPA。如要查看 API 的未來變更，請追蹤[這個 GitHub 存放區](#)。

若要整合範例應用程式 SPA 部分的共用元素轉場效果 API，請按照下列步驟操作：

1. 建立自訂 Hook，在 `use-page-transition.js` 檔案中管理轉場效果：

`utils/use-page-transition.js`


```
import { useEffect, useContext, useRef, useCallback } from 'react'
import ResourceContext from '../components/resource-context'
```

// Call this hook on this first page before you start the page transition. For Shared Element Transitions, you need to call the `transition.start()` method before the next page begins to render, and you need to do the Document Object Model (DOM) modification or setting of new shared elements inside the callback so that this hook returns the promise and defers to the callback resolve.

```
export const usePageTransitionPrep = () => {
  const { dispatch } = useContext(ResourceContext)

  return (elm) => {
    const sharedElements = elm.querySelectorAll('.shared-element')
    // Feature detection
    if (!document.createDocumentTransition) {
      return null
    }

    return new Promise((resolve) => {
      const transition = document.createDocumentTransition()
      Array.from(sharedElements).forEach((elm, idx) => {
        transition.setElement(elm, `target-${idx}`)
      })
      transition.start(async () => {
        resolve()
        await new Promise((resolver) => {
          dispatch({ type: 'update', transition: { transition, resolver } })
        })
      })
    })
  }
}
```

// Call this hook on the second page. Inside the `useEffect` hook, you can refer to the actual DOM element and set them as shared elements with the `transition.setElement()` method. When the resolver function is called, the transition is initiated between the captured images and newly set shared elements.

```
export const usePageTransition = () => {
  const { state, dispatch } = useContext(ResourceContext)
  const ref = useRef(null)
  const setRef = useCallback((node) => {
    ref.current = node
  }, [])

  useEffect(() => {
    if (!state.transition || !ref.current) {
      return
    }
    const { transition, resolver } = state.transition
    const sharedElements = ref.current.querySelectorAll('.shared-element')
    Array.from(sharedElements).forEach((elm, idx) => {
      transition.setElement(elm, `target-${idx}`)
    })
  })
}
```

```

    })
    resolver()
    return () => {
      dispatch({ type: 'update', transition: null })
    }
  })

  return setRef
}

```

2. 在清單頁面中呼叫 `usePageTransitionPrep()` 自訂 Hook，然後呼叫非同步函式，在 `click` 事件內觸發 `transition.start()` 方法。

在函式中，系統會收集 `shared-element` 類別元素，並註冊為共用元素。

components/list-item.js

```

// Codelab: Add the Shared Element Transitions API.
import { usePageTransitionPrep } from '../utils/use-page-transition'
...

function ListItemForSPA({ item, href }) {
  // Codelab: Add Shared Element Transitions.
  const transitionNextState = usePageTransitionPrep()
  const handleClick = async (e) => {
    const elm = e.target.closest('a')
    await transitionNextState(elm)
  }
  return (
    <Link href={href}>
      <a className='block flex items-center' onClick={handleClick}>
        <Icon src={item.image} name={item.name} className='shared-element' />
        <div className='text-xl'>{item.name}</div>
      </a>
    </Link>
  )
}

```

3. 在詳細資料頁面中，呼叫 `usePageTransition()` 掛鉤來完成 `transition.start()` 回呼函式。

在這個回呼中，詳細資料頁面中的共用元素也會註冊。

pages/fruits/[name].js

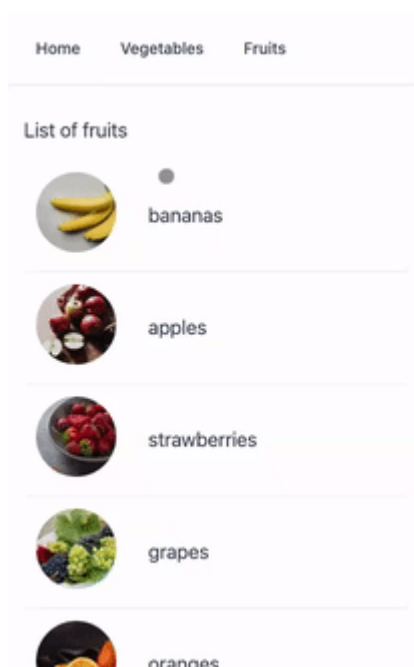
```
// Codelab: Add the Shared Element Transitions API.
import { usePageTransition } from '../utils/use-page-transition'

const Item = ({ data }) => {
  const { name, image, amountPer, nutrition } = data
  // Codelab: Add the Shared Element Transitions API.
  const ref = usePageTransition()

  return (
    <div className={'flex flex-col items-center justify-center py-4 px-4 sm:flex-row'} ref={
{ref}}>
      <div className='flex flex-col items-center sm:w-2/4'>
        <Image
          className='object-cover border-gray-100 border-2 rounded-full shared-element'
          src={image}
          width='240'
          height='240'
          alt={`picture of ${name}`}
        />
        <h1 className='text-4xl font-bold mt-4'>{name}</h1>
      </div>

      <div className='sm:w-2/4 w-full'>
        <Nutrition amountPer={amountPer} nutrition={nutrition} />
      </div>
    </div>
  )
  ...
}
```

現在您可以看到圖片元素在清單和詳細資料頁面中共用，並在頁面轉換中無縫連結。您甚至可以使用 [CSS 虛擬元素](#)，自訂動畫來增加精緻度。



[Home](#)[Vegetables](#)[Fruits](#)

List of fruits



bananas



apples



strawberries



grapes



oranges

7. 恭喜

恭喜！您建立的即時網頁應用程式流暢無阻，提供引人入勝的直覺式使用者體驗。

瞭解詳情

預先算繪

- [重新設計的預先算繪功能](#)
- [透過推測性預先算繪，讓瀏覽器即時載入網頁](#)
- [快速連結](#)

bfcache

- [往返快取](#)
- [往返快取測試工具](#)

跨網站預先擷取

- [Private Prefetch Proxy 說明](#)

Signed Exchange

- [已簽署的交換 \(SXG\)](#)
- [針對 Google 搜尋開始使用 Signed Exchange](#)

根/共用元素轉換

- [共用元素轉場效果](#)
- [使用 Shared Element Transitions API 實現流暢簡單的頁面轉場效果](#)

這些 API 仍處於開發初期，因此請在 crbug.com 分享意見回饋，或在相關 API 的 GitHub 存放區中提出問題。
